École de technologie supérieure — Eric Thé, Amal Ben Abdellah

INF-155 Travail Pratique #1 — Hiver 2026

Number Match

Travail en équipes de 2 ou 3

1 Objectifs

Ce travail pratique a pour objectifs de :

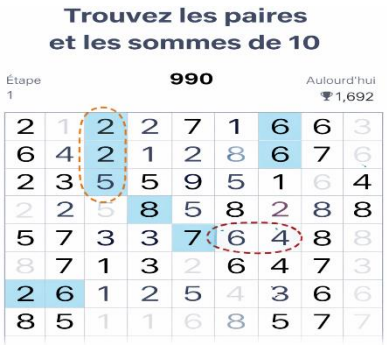
- Renforcer la maîtrise des notions fondamentales en programmation.
- Mettre en application les concepts relatifs :
  - aux pointeurs,
  - aux tableaux à une et deux dimensions,
  - aux macro-fonctions,
  - à la modularisation d'un programme.
- Respecter les bonnes pratiques de programmation, notamment :
  - une indentation claire et cohérente,
  - l'utilisation appropriée de constantes,
  - l'ajout de commentaires pertinents,
  - une structure de code lisible et organisée.

2 Description du problème : le jeu « Number Match »

« Number Match » est un jeu de casse-tête dont l'objectif est d'associer des chiffres identiques ou dont la somme est égale à 10 afin de les faire disparaître d'une grille. Ce jeu stimule la logique et la réflexion.

Les paires peuvent être :

- adjacentes (horizontalement, verticalement ou diagonalement),
- séparées par des cases vides,
- situées à la fin d'une ligne et au début de la ligne suivante (en passant par des cases vides).



Comment jouer

- L'objectif du joueur est de retirer tous les chiffres de la grille.
- Il faut repérer des paires de chiffres :
  - identiques (ex. 1 et 1, 7 et 7),
  - ou dont la somme est égale à 10 (ex. 6 et 4, 8 et 2).
- Les chiffres peuvent être :
  - voisins (horizontalement, verticalement ou diagonalement),
  - séparés par des cases vides,
  - situés à la fin d'une ligne et au début de la suivante.

- Le joueur sélectionne un premier chiffre, puis un second, afin de former une paire valide.

Attribution des points

- Deux chiffres situés dans des cases voisines donnent **1 point**.
- Deux chiffres séparés par des cases vides donnent **4 points**.
- Si tous les chiffres d'une ligne sont retirés :
  - la ligne est supprimée,
  - les lignes suivantes sont décalées vers le haut,
  - le joueur obtient **10 points supplémentaires**.

Règles supplémentaires

- Après le retrait d'une paire, si un chiffre n'apparaît plus dans la grille, il est retiré de la liste des chiffres disponibles.
  (Au début du jeu, les chiffres de 1 à 9 sont tous disponibles.)
- Lorsqu'aucun coup n'est possible :
  - le programme ajoute automatiquement de nouvelles lignes contenant des chiffres aléatoires en bas de la grille.
  - Ces chiffres sont choisis uniquement parmi ceux encore disponibles.
- Le joueur peut demander explicitement une régénération de nouveaux chiffres à tout moment. Cependant, un maximum de **trois régénérations** est permis par partie.
- Le jeu dispose également d'un système d'indices permettant d'aider le joueur lorsqu'il est bloqué.

Fin de la partie

- Le joueur gagne la partie lorsqu'il réussit à vider complètement la grille.
- La partie se termine si :
  - aucun coup n'est possible,
  - et qu'il ne reste plus de régénérations disponibles.

Plus la grille est vide, plus le joueur accumule de points.

3 Les tableaux nécessaires

La grille des chiffres est maintenue dans un tableau à deux dimensions d'entiers, avec le type défini suivant :

```
#define NB_COL  10      //10 = la colonne du nb. chiffres + 9 chiffres (colonnes [1] à [9])
#define MAX_LIG 20      //20 lignes de chiffres max. (à partir de [0])

//Type-tableau pour la grille de jeu (tableau 2D)
typedef int t_grille_nos[MAX_LIG][NB_COL];
```

Cependant, les valeurs de la première colonne du tableau (colonne [0]) ne sont PAS affichées à l'écran. En fait, cette colonne contient un compteur du nombre de chiffres sur cette ligne. Donc avec la grille illustrée à gauche, la représentation-tableau sera celle de droite (dans un tableau de type « `t_grille_nos` ») :

	A	B	C	D	E	F	G	H	I
1	2	3	4	5	6	7	7	3	1
2	4	5	1	6	1	7	3	8	1
3	9	2	3	4	5	6	7	1	3
4	1	4	5	8	1	8	1	2	3
5	5	6	3	4	8	1	6	1	8
6	2	4	5	6	7	1	3	1	4
7	6	6	1	8	1	2	6	5	6
8	3	4	5	8	1	2	5	3	7

Notez ici que :

1 – les cases VIDES sont représentés par un 0.

2 – la colonne [0] de la grille indique le nombre de chiffres sur cette ligne (cette colonne n'est pas affichée).

3 – les vraies colonnes de chiffres vont de [1] à [9], mais à l'écran les colonnes sont indexées de '**A**' à '**I**'.

4 – les lignes vont de [0] à [7] (avec ligne [19] max.), mais à l'écran les lignes sont indexées de 1 à 8.

	0	1	2	3	4	5	6	7	8	9
0	6	2	3	0	5	6	0	7	3	0
1	5	4	5	0	0	0	0	3	8	1
2	5	0	2	3	4	5	6	0	0	0
3	6	0	0	5	8	0	8	1	2	3
4	7	5	0	3	4	8	0	6	1	8
5	8	2	4	5	6	7	1	3	1	0
6	5	6	6	1	0	0	0	6	5	0
7	5	3	4	5	0	0	0	5	3	0

La liste des chiffres disponibles ('1' à '9' au départ) sera maintenue dans un tableau de 10 entiers :

```
//Type-tableau de la liste des chiffres disponibles (tableau 1D)
typedef int t_tab_chiffres[NB_COL];
```

Cependant, la valeur de la première case du tableau (case [0]) contient de nombre de chiffres encore visibles (ou disponibles) dans la grille de jeu. Voici l'état de cette liste pour la grille-exemple affichée plus haut :

0	1	2	3	4	5	6	7	8	9
8	6	4	9	5	8	7	2	5	0

Notez ici que :

1 – la case [0] du tableau contient 8 car il y a huit chiffres différents dans la grille (les '9' ont été éliminés!).

2 – les indices [1] à [9] correspondent aux chiffres '1' à '9' du jeu (eg. la case [**1**] indique qu'on a six **'1'**).

Donc, si on continue de jouer et que les deux chiffres '**7**' restants (dans les cases G1 et E6) sont éliminés de la grille, l'état du tableau des chiffres disponibles pourrait être le suivant (nous avons un chiffre en moins) :

0	1	2	3	4	5	6	7	8	9
7	6	4	**7**	5	8	7	**0**	5	0

Ces deux types-tableaux sont déclarés dans le fichier-interface « **grille_chiffres.h** » fourni avec cet énoncé.

4 La liste des coups possibles

Avant de faire la saisie du coup à jouer du joueur, on doit en premier lieu générer la liste de tous les couples de chiffres possibles à jouer. Ceci va permettre de valider que le coup joué par le joueur (indiqué par 2 coordonnées de cases de la grille) est bel et bien valide. La liste sera aussi utile lorsque le joueur demande un indice au programme, dans ce cas on va piger un des coups valides à jouer de la liste au hasard.

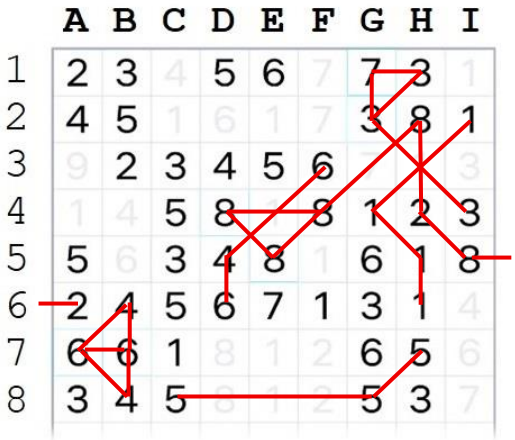
Voici le type-tableau utilisé pour conserver la liste des coups valides à jouer (dans « **liste_couples.h** ») :

```
#define MAX_COUPLES 200          //taille maximale de la liste
```

```
#define LIG_INFOS      0          //position-ligne pour les infos de la liste
#define COL_TAILLE     0          //position-colonne de la taille totale (à la ligne [0])
#define COL_NBELEM     1          //position-colonne du nb. d'éléments (à la ligne [0])

//Type-tableau pour la liste. Contient 2 colonnes pour les deux cases à jouer.
typedef int t_liste_couples[MAX_COUPLES + 1][2];   //eg. 24 - 34 (== "D3" - "D4")
```

Donc, selon la grille-exemple de la page 1, voici la liste des 23 couples de chiffres possibles à jouer :



	0	1			Points
0	**200**	**23**			
1	7	8	→	G1 - H1	1
2	7	17	→	G1 - G2	1
3	8	17	→	H1 - G2	1
4	17	39	→	G2 – I4	4
5	18	36	→	H2 - F4	4
6	18	38	→	H2 - H4	4
7	19	37	→	I2 – G4	4
8	26	44	→	F3 - D5	4
9	34	36	→	D4 - F4	4
10	34	45	→	D4 - E5	1
11	36	45	→	F4 - E5	1
12	37	48	→	G4 - H5	1
13	38	49	→	H4 – I5	1
14	44	54	→	D5 - D6	1
15	48	58	→	H5 - H6	1
16	49	51	→	I5 – A6	4
17	52	61	→	B6 - A7	1
18	52	62	→	B6 - B7	1
19	61	62	→	A7 - B7	1
20	61	72	→	A7 - B8	1
21	62	72	→	B7 - B8	1
22	68	77	→	H7 - G8	1
23	73	77	→	C8 - G8	4

Notez ici que :

1- La première ligne de la liste est réservée pour la taille de la liste et le nombre de coups (en rouge).

2- Au lieu d'utiliser 2 entiers pour indiquer une case (i.e. la ligne et colonne), nous utiliserons un seul entier où le chiffre des unités (chiffre de droite obtenu avec % 10) indique la colonne de la grille (de 1 à 9) et les autres chiffres (obtenus avec / 10) représentent la ligne de la grille (de 0 à 19 max.). Par exemple : **1** → case "**`A1`**", **9** → case "**`I1`**", **23** → case "**`C3`**", **91** → case "**`A10`**", **199** → case "**`I20`**"

3- La liste contient tous les coups possibles dans UN sens, pas les deux, donc si on ajoute le couple de cases "H2" à "F4" dans la liste nous n'allons pas en plus rajouter le couple inverse "F4" à "H2".

4- Le coup #14 qui va de "I5" à "A6" est un exemple de 2 chiffres qui se trouvent à la fin d'une ligne et au début de la ligne suivante. Notez que si la case "F3" était vide, nous aurions pu aussi ajouter le couple "E3" à "C4" (deux '5', de la fin de la ligne #3 à la ligne #4 en passant par des cases vides).

4.1 Votre premier mandat

Le module qui gère la liste des coups (**liste_couples.h**, **liste_couples.cpp**) vous est donné et vous devez compléter ce module en écrivant les 4 fonctions manquantes. NOTE : dans toutes ces fonctions, si on désire accéder aux valeurs de la ligne [0] de la liste, vous devez obligatoirement utiliser les constantes **LIG_INFOS**, **COL_TAILLE** et **COL_NBELEM** (eg. « liste[LIG_INFOS][COL_NBELEM] » plutôt que « liste[0][1] » pour obtenir le nombre actuel d'éléments dans la liste). Voir le code de « **init_liste_couples()** » qui est donné.

Ensuite, vous devez tester ces 5 fonctions avec un « *main()* » de tests unitaires en utilisant des « **assert()** ».

5 Deuxième mandat : Générer la grille initiale de chiffres

Au début du jeu on doit initialiser la grille de départ de façon aléatoire. Pour ce faire, on vous fourni la librairie « **utilitaires.h** ». Cette librairie offre trois fonctions. La première, « **init_rand()** », permet d'initialiser le générateur aléatoire pour garantir un comportement vraiment aléatoire d'une exécution à l'autre. Cette fonction doit être appelée qu'une seule fois, au tout début du « *main()* ». les deux autres fonctions servent à générer un nombre entier aléatoire ou un nombre réel aléatoire entre deux bornes reçues en paramètre.

Au départ, nous allons piger parmi les neuf chiffres possibles, de 1 à 9 car ils sont tous disponibles au début d'un jeu. Cependant, afin de ne pas offrir une grille trop facile à résourdre au départ, nous ne voulons pas que deux chiffres identiques soient générées dans deux cases adjacentes. En voici un exemple :

Notez ici que :

1 – un nombre aléatoire PAIR de chiffres doivent être générés (ici ce sont 42 chiffres au départ).

2 – les chiffres sont tous inscris de façon consécutive de gauche à droite, sans laisser une case vide.

3 – le restant de la dernière ligne générée (ainsi que toutes les lignes suivantes) sont des cases vides.

4 – aucun chiffres identiques se trouvent dans deux cases adjacentes lors de la génération initiale.

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0	9	2	3	9	5	6	1	7	9	2
1	9	4	5	7	8	9	8	3	8	1
2	9	7	2	3	4	5	6	9	6	5
3	9	6	8	1	2	3	7	5	4	9
4	6	4	1	5	8	6	4	0	0	0

← la dernière ligne de chiffres générée.

Liste des chiffres disponibles :

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
9	4	4	4	5	6	5	4	5	5

5 – les quantités de chaque chiffre disponible dans le tableau de droite doivent donner la somme de 42.

6 – après la génération de chiffres, la fonction d'initialisation va retourner le numéro de la dernière ligne.

5.1 La sous-fonction « verifier_ch_voisins() »

Afin de vérifier qu'un chiffre n'apparaisse pas deux fois dans un même voisinage, vous devez développer la sous-fonction « verifier_ch_voisins() ». Cette fonction reçoit la grille des chiffres, la position-ligne et colonne actuelle et un chiffre à insérer dans la grille. Elle va retourner un entier. La fonction doit simplement valider que le chiffre reçu n'apparait pas dans les cases directement adjacentes à la case [ligne, colonne].

Par exemple, avec la grille illustrée plus haut, si nous voulons ajouter un nouveau chiffre dans la case [4, 7] ce nouveau chiffre ne doit pas être un '**4**', '**5**' ou un '**7**'. Si le chiffre reçu est un de ces chiffres on retourne **0**.

NOTE : la fonction sera une fonction privée de « **grille_chiffres.cpp** » (ne pas la déclarer dans le fichier *.h).

5.2 La fonction d'initialisation de la grille

La fonction d'initialisation de la grille est déjà déclarée dans le fichier-interface « **grille_chiffres.h** ». Cette fonction initialisera la grille ainsi que la liste des chiffres disponibles. Elle doit premièrement choisir le nombre initial de chiffres à générer. On détermine un facteur réel aléatoire entre [4.0, 5.0], ce facteur est ensuite multiplié par 9 (le nombre de chiffres au départ) et si l'entier ainsi obtenu est impair y rajouter un +1. Ceci nous donne le nombre initial de chiffres à générer.

On débute la génération de chiffres à partir de la position [0, 1] (car la colonne [0] contient le nombre de chiffres sur chaque ligne). Chaque chiffre est choisi au hasard avec la fonction « entier_aleatoire() », on doit valider que ce chiffre n'apparait pas dans le voisinage immédiat de la position actuelle [lig, col]. Sinon, en choisir un autre. Le chiffre choisi est ensuite inscrit dans la case actuelle et on incémente la position actuelle vers la doite. Notez que quand on change de ligne, la position-colonne doit être remise à 1 (pas [0]).

MACROS : la mise-à-jour de la position doit être fait avec une macro-fonction « `INC_POS(lig, col)` ».

On doit aussi incrémenter le nombre d'occurrences de chaque chiffre de la liste « nbr_chiffres[ ] ». N'oubliez pas le compteur [0] de cette liste! On répète le tout selon le nombre initial de chiffres déterminé au départ.

NOTE : lorsque l'on veut consulter ou modifier un des compteurs-ligne de la grille (la colonne [0]) ou de la liste des chiffres disponibles (la case [0]), vous devez absolument utiliser la constante **POS_NB** (pas [0]) !

Après que tout les chiffres ont été générées il se peut qu'un chiffre parmi les 9 (ou plus) n'aura jamais été sélectionné! Donc avec la liste « nbr_chiffres[] », si la case [0] de cette liste n'est pas à 9, il manque un (ou plusieurs) chiffre(s). Trouver le(s) chiffre(s) manquant et le(s) rajouter à la suite des chiffres générés.

La fonction se termine en retournant la position-ligne du dernier chiffre généré (ci-haut la ligne #4).

TEST : Afin de tester l'initialisation de la grille, déclarez les deux tableaux principaux dans votre « *main()* », Faites appel à « init_rand() » et ensuite « init_grille() ». Vérifier ensuite le contenu de la grille générée ainsi que l'état des 9 compteurs dans la liste des chiffres disponibles.

6 Le module d'affichages

Le module d'affichages vous est partiellement fourni. À vous d'écrire les fonctions manquantes de ce module. Commencez par écrire les deux fonctions « **afficher_infos_jeu()** » et « **afficher_grille()** ». La fonction « afficher_infos_jeu() » affichera le pointage actuel du jeu plus-ou-moins centré à la première ligne de l'écran suivi de la liste des chiffres encore disponible à la ligne-écran suivante. Si un des 9 chiffres n'est plus disponible (i.e. son compteur est à zéro), on affiche un espace au lieu du chiffre. Vous devez absolument utiliser les constantes de position-lignes et colonnes déclarées au début du fichier-interface « affichages.h ».

La fonction « afficher_grille() » affichera les lignes non-vides de la grille à partir de la position-écran (**COL_GRILLE, LIG_GRILLE**). La première ligne affichée ici est la ligne des indices-colonne de la grille (de '**A**' à '**I**') affiché en JAUNE. Chaque indice affiché sera précédé d'un espace (avec 2 caractères par position).

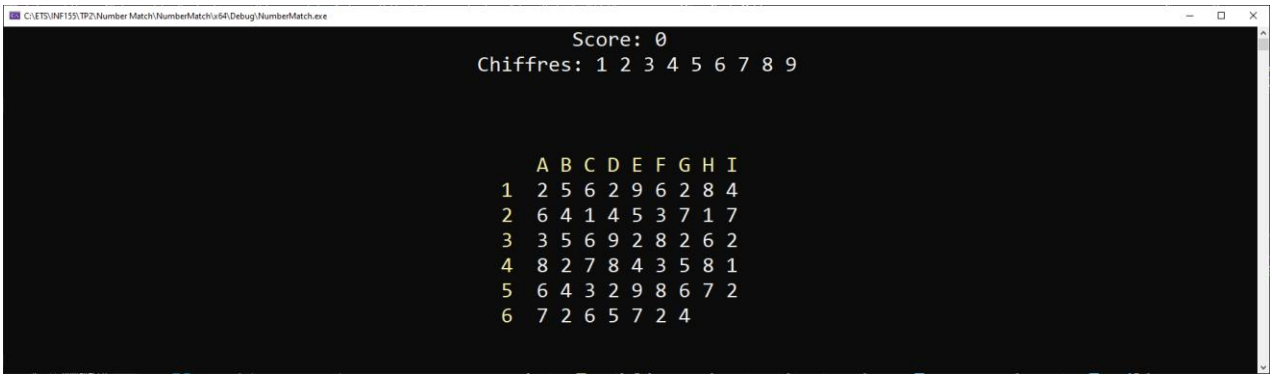
On affiche ensuite tous les chiffres des lignes non-vides de la grille. Chaque ligne débute avec son indice (à partir de '**1**', pas '0') affiché en JAUNE. Viennent ensuite les chiffres des cases de la grille, précédés d'un espace. Si une case de la grille est vide (i.e. contient un 0), on affichera deux espaces pour cette case.

Module d'affichage « WinConsole »

Pour permettre l'affichage d'un message ou un chiffre à une position (x, y) de l'écran, avec choix de couleurs, nous vous fournissons un module (**winconsole.h** et **winconsole.cpp**) qu'il faut inclure dans votre projet. Sachez que les coordonnées en x sont définies de [0 à 119] et les coordonnées en y sont définies de [0 à 24] dans la console-écran. Les détails pour inclure et utiliser ce module vous seront donnés en classe, mais sachez que les principales fonctions qui seront utiles sont : *clrscr(), gotoxy(), textcolor(), clreol()* et *delay().*
IMPORTANT : le fichier « **affichages.cpp** » sera le SEUL à inclure le fichier « winconsole.h » dans le projet.

TEST : Dans votre « *main()* », suite à la génération initiale de la grille, faites afficher le pointage de départ et la liste des chiffres disponibles sur les deux premières lignes de l'écran, suivi de l'affichage de la grille.

Voici un exemple de ce que les affichages de départ devraient donner dans la console :



Notez que les lignes vides situées en dessous de la liste des chiffres disponibles sont réservées pour la saisie des cases à retirer (à la LIG_SAISIE) et pour l'affichage de différents messages d'erreur ou d'information (à la LIG_MESSAGES).

7 Génération de la liste des coups possibles

L'étape suivante de développement est d'extraire la liste de tous les coups possibles à partir de la grille de départ. Nous aurons besoin de deux autres sous-fonctions privées dans le fichier « **grille_chiffres.cpp** ».

7.1 La sous-fonction « verifier_huit_directions() »

Cette sous-fonction reçoit la grille des chiffres, une position-ligne et colonne actuelle et le tableau des coups possibles. Elle retourne rien (*void*).

Pour chacune des 8 directions possibles de la rose-des-vents, on se déplace dans cette direction à partir de la position actuelle [lig, col] afin de trouver la première case non-vide dans cette direction (peut être la case voisine de [lig, col] ou plus loin). Si on atteint le bord de la grille (vers le haut ou la gauche) ou si on atteint une ligne complètement vide (vers le bas), on doit arrêter ce déplacement (le cas spécial du déplacement vers la droite sera traité ailleurs, voir la section 7.2).

Si la case finale [lig2, col2] après un déplacement est est non-vide et toujours dans la grille, on vérifie si le chiffre de cette case est identique au chiffre de la case [lig, col] OU complète une somme de 10 avec ce chiffre. Si c'est le cas, et que le couple [lig, col] – [lig2, col2] n'existe pas dans la liste des coups, on l'ajoute.

7.2 La sous-fonction « verifier_sens_est() »

Cette deuxième sous-fonction privée à rajouter aura la tâche de faire la recherche des coups possibles dans le cas spécail du déplacement vers la droite (le sens EST). Elle reçoit la grille des chiffres, une position-ligne et colonne actuelle ainsi que le tableau des coups possibles et retourne rien (*void*).

On se déplace vers la droite à partir de la position actuelle [lig, col] afin de trouver la première case non-vide dans cette direction. Lorsque l'on dépasse le bord de la grille vers la droite (i.e. colonne [10]), on se déplace au début de la ligne suivante (à la colonne [1]), et on continue d'avancer afin de trouver une case non-vide. NOTE : la mise-à-jour de la position doit être fait avec la macro-fonction « **`INC_POS`**`(lig2, col2)` ».

Si la case finale [lig2, col2] après ce déplacement est est non-vide et toujours dans la grille, on vérifie si le chiffre de cette case est identique au chiffre de la case [lig, col] OU complète une somme de 10 avec ce chiffre. Si c'est le cas, et que le couple [lig, col] – [lig2, col2] n'existe pas dans la liste des coups, on l'ajoute.

Cette sous-fonction sera appelée à partir de « verifier_huit_directions() », dans le cas spécial de la recherche vers la droite (sens EST). Donc les déplacements dans cette direction sont différents des 7 autres directions.

7.3 Fonction principale de recherche des coups possibles

La fonction principale de recherche des coups possibles, « **generer_listes_couples()** », reçoit la grille de jeu, l'indice de la dernière ligne non-vide de la grille ainsi que la liste des coups possibles, initialement initialisée à l'état vide. Elle retourne un entier.

Simplement parcourir toutes les lignes non-vides de la grille. Pour les 9 colonnes de chaque ligne, si la case actuelle est non-vide, on fait appel à « verifier_huit_directions() » pour ajouter tous les coups possibles à partir de la case [lig, col] actuelle. La fonction se termine en retournant le nombre total de coups ajoutés.

7.4 Votre troisième mandat

Dans votre « *main()* », ajouter la déclaration du tableau pour la liste des coups possibles. Suite à la génération initiale et l'affichage de la grille, faites appel à la fonction « init_liste_couples() » suivi de « generer_liste_couples() ». Vérifier ensuite le contenu de la liste des coups générée avec la grille initiale.

AIDE : au lieu de générer aléatoirement la grille de départ, vous pouvez y copier toutes les valeurs de la grille-exemple de la page 2 de cet énoncé (eg. « t_grille_nos grille = { 6, 2, 3, 0, 5, 6, 0, 7, 3, 0, ...etc }; »), et ensuite valider que la liste des coups contienne exactement les mêmes 23 coups affichés à la page 3.

8 Les autres fonctions du module « grille_chiffres »

Il est maintenant temps de compléter toutes les autres fonctions du module « **grille_chiffres.h** » (SAUF la fonction « ajouter_chiffres() », voir la section 10.2). Voici les détails importants concernant ces 5 fonctions :

retirer_ligne() – on reçoit l'indice « *no_lig* » d'une ligne vide de la grille qui devra être retirée de la grille. À partir de cette ligne, copier les 10 valeurs (incluant la case-compteur) de la ligne suivante sur cette ligne et ensuite incrémenter le « *no_lig* » courant. On répète le tout jusqu'à ce qu'on ai copié une ligne vide complète de la grille OU que l'on à copié le contenu de la dernière ligne possible (la ligne [19]). Nous aurons maintenant une ligne de chiffres en moins dans la grille de jeu.

effacer_chiffre() – on reçoit une position-case « *pos* » de la grille que l'on doit vider. Donc simplement obtenir la position-ligne et colonne de la grille à partir de « *pos* » et inscrire un zéro (0) dans cette case de la grille. N'oubliez pas de décrémenter le compteur du nombre de chiffres sur cette ligne (la colonne [0]) !

retirer_chiffre() – on reçoit un chiffre « *ch* » (qui vient d'être effacé) ainsi que la liste des chiffres disponibles. Simplement décrémenter le compteur associé à ce chiffre dans le tableau. Si ensuite ce compteur tombe à zéro, on décrémente le nombre de chiffres restants dans la case [0] de la liste et on retourne le chiffre « *ch* ». Sinon, on retourne 0 (signifie qu'aucun chiffre a été éliminé de la liste des chiffres disponibles).

get_chiffre_case() – fonction-accesseur au chiffre se trouvant dans la case « *no_case* » de la grille de jeu.

nb_chiffres_restants() - fonction-accesseur au nombre de chiffres du jeu encore disponibles (la case [0]).

➔ Vous devez tester rigoureusement ces 5 fonctions (dans votre « *main()* » de tests) avant de continuer !

9 Le module de gestion du déroulement du jeu

C'est votre tâche d'écrire ce dernier module du programme au complet (fichiers *.h et *.cpp). Ce module doit inclure que le fichier « **affichages.h** ». Voici les constantes à déclarer dans son fichier-interface (*.h) :

```
/* Les 3 valeurs de codes-commandes (peuvent aussi être déclarées dans une liste "enum") */
#define CODE_CHIFFRES -2        //demander une régénération de nouveaux chiffres
#define CODE_AIDE     -1        //demander une suggestion d'un coup à jouer
#define CODE_QUITTER   0        //code pour quitter le jeu

/* Les valeurs des points attribués durant le jeu */
#define PTS_COUPLE_VOISIN  1    //élimination d'un couple de chiffres voisins (distance de 1)
#define PTS_COUPLE_SEPARE  4    //élimination d'un couple de chiffres séparés par des espaces
#define PTS_LIGNE_RETIREE  10   //élimination d'une ligne vide suite à un retrait de chiffres
```

Ce module va définir 3 fonctions publiques et une fonction privée. De plus, afin de faire les conversions nécessaires du format de saisie d'une adresse-case à une position-case entière (eg. de "B2" à l'entier 12), vous devez définir et utiliser les trois MACRO-fonctions suivantes (dans le fichier-implémentation *.cpp) :

```
1 - CHIFFRE_A_VAL(ch)   //conversion d'un caractère-chiffre '0' (char) à sa valeur numérique 0 (int)
2 - LETTRE_A_COL(let)   //conversion d'une lettre 'A' (char) à l'indice-colonne équivalente 1 (int)
3 - COL_A_LETTRE(col)   //conversion d'un indice-colonne 1 (int) à la lettre équivalente 'A' (char)
```

Voici maintenant la description des premières fonctions à implémenter dans ce module.

9.1 La sous-fonction « saisie_case() »

Cette sous-fonction privée au module reçoit l'indice de la dernière ligne active de la grille et le « *numéro* » de la saisie (soit 1 ou 2). Elle retournera un résultat entier. Cette fonction effectue la saisie et la validation d'une case valide (eg. "A1") OU d'un code-commande (avec soit "??", "++" ou "XX").

Dans une boucle de validation, on affiche le message de sollicitation pour la saisie d'une case avec le « *numéro* » de saisie actuel. Par exemple (avec '2') : `Entrer la case 2 (eg. A1): _`

Utilisez la fonction « **demander_saisie()** » du module « affichages » qui affiche le message de sollicitation avec ce numéro à la LIG_SAISIE de l'écran et qui efface le restant de cette ligne de saisie (avec "clreol()").

On effectue ensuite la lecture d'une chaîne de caractères. Si cette chaîne est un "**XX**" (ou "**xx**"), la fonction retourne immédiatement le **CODE_QUITTER**. Si cette chaîne est la chaîne "**??**" on retourne le **CODE_AIDE.** Et si cette chaîne est la chaîne "++" on retourne le **CODE_CHIFFRES.** Sinon, c'est probablement l'adresse d'une case de la grille, que l'on doit valider.

Premièrement, la chaîne saisie doit être de soit **2 ou 3** caractères au total ET le premier caractère de la chaîne doit être une lettre '**A**' à '**I**' (majuscule OU minuscule), sinon c'est une saisie non-valide. Voici quelques exemples de chaînes-adresses valides : "**a1**", "**A1**", "**B10**", "**I20**" (la plus grande adresse possible).

Si la chaîne semble être valide, le deuxième caractère devrait être un chiffre. On obtient la valeur numérique correspondante de ce chiffre (valeur de 1 à 9), et si la chaine contient un 3ième caractère (un autre chiffre), obtenir la valeur numérique de ce chiffre et l'ajouter comme le chiffre des unités d'un indice de rangée > 9 (eg. "**A1**" → colonne = '**A**' et rangée = **1**. "**A12**" → colonne = '**A**' et rangée = 10 + 2 = **12**). On décrémente ensuite de -1 la rangée obtenue, car les vrais indices des rangées débutent à partir de [0] (pas [1]). Si le vrai numéro de ligne ainsi obtenu est plus grand que la dernière ligne active de la grille, la ligne est invalide.

Sinon, on multiplie cette ligne par 10 et on y ajoute l'indice-colonne correspondant à la lettre du premier caractère (eg. avec "**A12**" on obtient (11 * 10) + 1 → position **111.** Avec "**C1**" on obtient (0 * 10) + 3 → **3**). Ceci est la position-case équivalente à retourner. Notez que les positions 10, 20, 30, etc. sont NON-valides car la colonne [0] de chaque ligne ne fait pas partie de la grille des chiffres à retirer.

On doit donc saisir une chaîne tant et aussi longtemps qu'elle est jugée non-valide. Si la chaîne est jugée non-valide, vous devez afficher un message d'erreur explicatif (utilisez la fonction « **mess_erreur()** » du module « affichages » qui affiche un message d'erreur en rouge à la LIG_MESSAGES de l'écran avec un effet de clignotement). Lorsque jugée valide, la boucle s'arrête et la fonction retourne soit un code-commande ou une position-case valide (un entier).

9.2 La fonction « valider_coup() »

La sous-fonction « saisie_case() » sera utilisée dans cette première fonction publique du module. Elle reçoit la liste des coups possibles, la dernière ligne active de la grille et 2 pointeurs à des entiers « *caseA, caseB* ». La fonction retournera soit **1** (un couple de cases valides a été saisi), ou **0** (saisie d'un code-commande).

Voici la stratégie à implémenter dans cette fonction qui va saisir un coup valide à jouer du joueur :

- **Répéter** :
  - *caseA* ← Demander la saisie de la première case du couple
  - Si « *caseA* » a reçu soit le **`CODE_QUITTER`**, **`CODE_AIDE`** ou **`CODE_CHIFFRES`**,
    - Quitter la fonction immédiatement avec un retour de **0**.
  - *caseB* ← Demander la saisie de la deuxième case du couple
  - Vérifier si le couple (*caseA, caseB*) existe déjà dans la liste des coups possibles,
  - Si ce couple n'est PAS dans la liste,
    - Afficher un message d'erreur disant que ce coup est non-valide.
- **Tant Que** : le couple saisi (*caseA, caseB*) n'est pas dans la liste des coups possibles.
- Retour de **1** (un couple valide a été saisi et retourné dans les deux références *caseA* et *caseB*)

9.3 La fonction « jouer_coup() »

Ceci est la fonction principale de gestion du jeu. Elle reçoit la grille de jeu, deux entiers (*caseA, caseB*) représentant deux cases à vider, le tableau des chiffres disponibles et un pointeur à la dernière ligne active. Elle retourne un résultat entier, soit le nombre de points obtenus après avoir joué ce coup dans la grille.

1) Déclarer deux compteurs « *points* » et « *pts_lignes* » initialisés à zéro.
2) Si la distance linéaire absolue entre la « *caseA* » et la « *caseB* » est égale à 1,
   - Ajouter +1 point aux « *points* » à date et afficher un message indiquant ce gain de +1 point (avec la fonction « **mess_points()** » du module « affichages » qui affiche un message de gain de points en vert à la LIG_POINTS de l'écran avec un effet de clignotement)
3) Sinon (les deux cases sont séparées par des cases vides),
   - Ajouter +4 points aux « *points* » à date et afficher un message indiquant ce gain de +4 points
4) Obtenir le chiffre qui se trouve dans la « *caseA* » de la grille et retirer une occurrence de ce chiffre dans la liste des chiffres disponibles.

5) Si la fonction de retrait du chiffre a retourné ce même chiffre (plutôt que 0), afficher un message indiquant que ce chiffre vient d'être éliminé du jeu.

6) Effacer ce chiffre de la « *caseA* » de la grille.

7) Répéter les étapes 4, 5, et 6 pour le chiffre de la « *caseB* » de la grille.

8) Si (*caseA* < *caseB*), // on veut que « *caseA* » et « *caseB* » soient en ordre décroissant!
   - permuter ces 2 valeurs.

9) Si la ligne de la grille donnée par la position « *caseA* » est maintenant VIDE,
   - Retirer la ligne de la « *caseA* » de la grille et ajouter +10 points au compteur « *pts_lignes* »
   - Décrémenter la valeur du pointeur de la dernière ligne active (la grille à une ligne en moins).

10) Si la ligne donnée par la « *caseB* » est différente que celle de la « *caseA* » ET qu'elle est VIDE,
   - Retirer la ligne de la « *caseB* » de la grille et ajouter +10 points au compteur « *pts_lignes* »
   - Décrémenter la valeur du pointeur de la dernière ligne active (la grille à deux lignes en moins).

11) Si on a obtenu des « *pts_lignes* », afficher un message qui indique le nombre de lignes retirées ainsi que le total de points bonis obtenus (soit +10, ou +20 pour 2 lignes retirées).

12) Retourner la somme des « *points* + *pts_lignes* » obtenus.

9.4 Quatrième mandat : la première version du « main() » final

Vous êtes maintenant prêts à implémenter la boucle principale de déroulement du jeu dans votre « *main()* ». Dans votre fichier-principal, vous n'avez qu'à inclure le fichier-interface (*.h) du module de gestion du jeu.

<u>Variables-tableaux :</u> la grille de jeu, la liste des coups possibles, la liste des chiffres disponibles initialisé à 0.

<u>Variables entières :</u> dernière_lig, caseA, caseB, score = 0, quitter =0

- Faites appel à « *init_rand()* » et ensuite « *init_grille()* » et récupérer la dernière ligne active de la grille
- Faites afficher le pointage de départ et la liste des chiffres disponibles, suivi de l'affichage de la grille.
- **Répéter** :
  - Faites appel à « init_liste_couples() »
  - Si l'appel de « generer_liste_couples() » a retourné un nombre > 0 (des coups sont possibles),
    - Faites appel à « valider_coup() »,
    - **SI** cette fonction a retourné 1,
      - Exécuter ce coup et ajouter les points obtenus au « *score* » du jeu actuel.
    - **Sinon**, (la fonction "valider_coup()" a retourné un 0),
      - Quitter la boucle principale (pour l'instant, les 3 codes-commandes vont arrêter le jeu. Cette partie du code sera bonifié dans les étapes 10.3 et 11.3 du travail)
  - Fin Si
  - Faites afficher le score à date et la liste des chiffres disponibles, suivi de l'affichage de la grille.
- **Tant Que** : le joueur n'a pas choisi de quitter, qu'il reste des chiffres disponibles et des coups à jouer.

- Si le joueur n'a pas choisi de quitter,
  - Faites afficher un message de fin avec le score final du jeu.

Votre devriez maintenant être capable de jouer le jeu normalement, jusqu'à ce qu'il ne reste plus de coups possibles à jouer dans la grille. Pour l'instant, la saisie des chaines « XX », « ++ » ou « ?? » pour la première case va faire arrêter la boucle. Testez aussi la saisie de coups non-valides ou non-présents dans la liste.

10 Ajout de la régénération de chiffres

Lorsqu'il ne reste plus de coups possibles à jouer dans la grille, le programme va automatiquement générer une nouvelle série de chiffres aléatoires à la fin de la grille. Cette régénération de chiffres est aussi disponible sur demande du joueur, avec la saisie de la chaine « **++** ». Durant le jeu, seulement 3 régénérations de nouveaux chiffres seront permises.

Voici les ajouts et modifications à apporter au programme pour implémenter cette nouvelle fonctionnalité.

10.1 Le sous-programme « trouver_fin_chiffres() »

Dans le fichier « grille_chiffres.cpp », ajoutez une sous-fonction privée « **trouver_fin_chiffres()** ». Cette fonction reçoit la grille de jeu et deux références à une « *ligne* » et une « *colonne* ». La référence-ligne sera déjà initialisée avec la dernière ligne active de la grille. Cette fonction retournera rien (*void*).

Initialiser la valeur de la référence-colonne à 1. Ensuite, avec une boucle, parcourir la ligne reçue de la grille de gauche à droite, en comptant les chiffres non-zéro de cette ligne. Une fois que vous aurez compté le même nombre de chiffres non-zéro que le compteur inscrit dans la colonne [0] de cette ligne, on arrête. La case suivante sera la première case vide après le dernier chiffre de la grille (eg. avec la grille-exemple de la page 2, la première case vide est la case [7, 9] ou "`I8`"). Les coordonnées (*lig, col*) de cette case vide seront renvoyées via les deux références reçues. Faites attention au cas spécial où la première case vide se trouve au début de la ligne suivante (i.e. la dernière case de la dernière ligne active est remplie) !

10.2 La fonction « ajouter_chiffres() »

Écrire maintenant la définition de la fonction « **ajouter_chiffres()** », déjà déclarée dans « grille_chiffres.h ». Elle reçoit la grille de jeu, la listes des chiffres disponibles et une référence à la dernière ligne active. Tout comme lors de la génération initiale de la grille, le nombre de chiffres à générer dépends d'un facteur réel.

On détermine un facteur réel aléatoire entre [MIN_NB_NOUV, MAX_NB_NOUV] (soit [1.5 à 5.0]), ce facteur est ensuite multiplié par le nombre de chiffres encore disponibles et si l'entier ainsi obtenu est impair y rajouter un +1. Ceci nous donne le nombre de nouveaux chiffres à générer.

Faites ensuite appel à « trouver_fin_chiffres() » afin d'obtenir la coordonnée de départ pour la régénération des chiffres. Chaque chiffre est choisi au hasard avec la fonction « entier_aleatoire() », on doit valider que ce chiffre n'est <u>pas le même que chiffre précédent</u> dans la grille (on ne doit PAS vérifier toutes les cases voisines de la case actuelle ici). Sinon, en choisir un autre (sauf si il ne reste qu'UN seul chiffre disponible). Le chiffre choisi est ensuite inscrit dans la case actuelle et on incémente la position actuelle vers la droite.

On doit aussi incrémenter le nombre d'occurrences de chaque chiffre de la liste « nbr_chiffres[ ] ». Faites attention de ne pas déborder des 20 lignes max. de la grille! La fonction retournera rien, mais l'indice de la dernière ligne active de la grille sera modifiée via la référence à cette variable du « *main()* ».

10.3 Modifications au « main() » du jeu

Ajoutez deux variables entières « *regen* = **0**, *nb_regen* = **3** » dans votre « *main()* ». Ces variables indiquent si l'on doit régénérer de nouveaux chiffres dans la grille ou non et le nombre de régénérations restantes.

Maintenant, dans le bloc de code associé à la partie « **Sinon**, (la fonction "valider_coup()" a retourné un 0) » du « *main()* » de la section 9.4, on ajoute une condition pour savoir si « *caseA* » a reçu le **CODE_CHIFFRES**. Si oui, et que le « *nb_regen* » n'est pas à zéro, on mets « *regen* = **1** ». Sinon, on affiche un message indiquant qu'il ne reste plus de régénaérations possibles. Ensuite, après le « Fin Si » de ce « *main()* » (avant les affichages de fin de boucle), on insère le code suivant :

- **SI** (*regen* = 1 OU que la liste des coups possibles est vide ET qu'il reste au moins une régénération),
    - Faites appel à la fonction d'ajout de nouveaux chiffres.
    - Faites appel à « init_liste_couples() » et ensuite « generer_liste_couples() ».
    - Remettre « *regen* » à zéro.
    - Décrémenter de -1 le « *nb_regen* ».
    - Afficher un message qui indique le nombre de régénérations restantes dans le jeu
- **Fin Si**

11 Ajout de la demande d'un indice

La dernière option de jeu à implémenter est la demande d'un indice au programme avec la saisie de la chaîne « **??** ». Dans ce cas, le programme va choisir aléatoirement un coup parmi ceux inscrits dans la liste des coups possibles et affichera cette suggestion de deux cases à jouer. Le choix aléatoire du couple de cases à jouer sera effectué avec la fonction « **choix_couple_alea()** » du module « **liste_couples** ».

Voici les ajouts et modifications à apporter au programme pour implémenter cette nouvelle fonctionnalité.

11.1 La fonction d'affichage « afficher_couple() »

Dans le module « affichages », écrire la fonction « **afficher_couple()** ». Elle reçoit deux positions de cases « *colA, ligA* » et « *colB, ligB* » ainsi que les deux chiffres « *chA, chB* » qui se trouvent dans ces deux cases. Simplement se positionner à l'écran selon les coordonnées « *colA, ligA* » et « *colB, ligB* » et afficher en BLEU les deux chiffres « *chA* » et « *chB* » pendant 5 secondes. Remettre ensuite la couleur d'affichage à BLANC.

11.2 La fonction « afficher_indice() »

C'est la fonction principale d'affichage de l'indice proposé par le programme. Elle sera ajoutée à votre module de gestion du jeu. Elle reçoit la grille de jeu ainsi que les deux cases « *caseA, caseB* » du coup suggéré par le programme et elle retournera rien (*void*). On récupère les deux chiffres qui se trouvent dans la grille aux deux cases reçues. On construit ensuite une chaine qui indique les deux cases à jouer avec les numéros de cases converties en positions au format « A1 ». Par exemple, avec les cases **24** et **25**, le message sera « *Jouez les cases **D3** et **E3*** »). On affiche ce message (à la ligne de messages habituel) et on fait ensuite appel à « afficher_couple() » pour voir les deux cases suggérées en BLEU.

11.3 Modifications au « main() » et derniers affichages

Dans le bloc de code associé à la partie « **Sinon**, (la fonction "valider_coup()" a retourné un 0) » du « *main()* » de la section 9.4, on ajoute une condition pour savoir si « *caseA* » a reçu le **CODE_AIDE**. Si oui, on fera simplement appel à « choix_couple_alea() » suivi de « afficher_indice ».

Votre programme est maintenant complet au niveau des fonctionnalités demandées. Il vous reste à rajouter quelques fonctions d'affichages supplémentaires (dans le module « affichages ») tels que :

- **afficher_instructions()** : fait les affichages de départ du jeu, indiquant les règles du jeu et les options.
- **afficher_fin_jeu()** : fait les affichages finaux après la fin du jeu en indiquant le pointage total obtenu.

Le but ici et de faire AUCUNS appel à « *printf()* » (ou à la fonction « *gotoxy()* » de « WinConsole ») dans le code du « *main()* ». Donc, tous les affichages sont encapsulés dans les fonctions du module « affichages ».

BONUS : pour quelques points bonis, vous pouvez ajouter la détection d'une grille complètement vidée à la fin jeu, après la boucle principale. Dans le vrai jeu, cette réussite ajoute **+140** points au pointage total.

12 Contraintes de l'enseignant

- Aucune variable globale ne sera acceptée – L'utilisation d'une seule variable globale entraine une pénalité automatique de 25%.
- L'utilisation de l'instruction `GOTO` est interdite – L'utilisation de cette instruction entraine une pénalité automatique de 25%.
- Vous devez obligatoirement remettre un programme offrant les mêmes fonctionnalités que le programme exécutable fourni sur Moodle avec cet énoncé.
- La remise se fait par un dépôt sur Moodle. Le site sera configuré **pour refuser tout travail en retard**. Ainsi, aucun travail ne pourra être remis en retard et la note de 0 sera attribuée.

BON TRAVAIL ! ☺